BPUFuzzer: Effective Fuzz Testing for Branching Transient Execution Vulnerabilities of RISC-V CPU

Rihui Sun[1], Jin Wu[1], Hanyin Liu[1], Zikang Tao[1], Gang Qu[3], Dongsheng Wang[2], Yongqiang Lyu[2], Jian Dong[1*]
[1]Harbin Institute of Technology, Harbin, Heilongjiang, China
[2]Tsinghua University, Beijing, China
[3]University of Maryland, College Park, MD, US

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133085

***Abstract*—This paper presents BPUFuzzer, a fuzz testing tool for detecting branching transient execution vulnerabilities in CPU RTL design. BPUFuzzer addresses two key challenges: generating testcases that capture complex control flows, and extracting essential data from vast hardware states to guide testcase selection. Utilizing a control flow graph-based testcase generation strategy with anomaly detection and employing fitness and coverage metrics, BPUFuzzer works on testcases that cover broader program flows and deliberately selects testcases to discover transient execution vulnerabilities effectively. When applied on RISC-V Boom v3, BPUFuzzer uncovered more Spectre types than the state-of-the-arts, including a previously unidentified variant, named Spectre-LOOP.**

I. Introduction

Transient execution vulnerabilities are a significant hardware security issue in modern CPUs. Attacks like Spectre [1] and Meltdown [2], along with their variants [3]–[11], severely impact the reliability of processor hardware. The majority of these vulnerabilities are caused by the malicious exploitation of the branch prediction unit (BPU), which is a commonly implemented microarchitecture in modern processors designed to optimize pipeline performance by predicting control flow changes thus preventing pipeline stalls [12]. The discovery of these vulnerabilities and their variants has raised new concerns about processor security, as these microarchitecture-based issues cannot be easily fixed by vendors and pose serious threats to information system security.

Recent advancements have been made in the automated detection of transient execution vulnerabilities. Several approaches [13]–[17] focus on testing limited code snippets based on prior knowledge, without addressing a broad range of code patterns. SpecDoctor [18] generates testcases without prior knowledge but imposes constraints that prevent loop generation in its implementation. Furthermore, it lacks coverage feedback mechanisms specific to transient execution vulnerabilities, limiting its ability to detect issues arising from complex hardware states. Moreover, current techniques limit their vulnerabilities detection without in-depth analysis of complex BPU behaviors. Detecting vulnerabilities within BPU poses two significant challenges. First, since the combination of branch instructions that can change the BPU state is very diverse, covering a broad range of control flow patterns is difficult. Second, it is challenging to efficiently search for testcases that tend to trigger transient execution from numerous testcase patterns corresponding to complex BPU states.

In this paper, we present BPUFuzzer, a pre-silicon fuzz testing tool designed to address these challenges and effectively detect branching transient execution vulnerabilities. In order to capture complex control flow patterns, we propose an effective testcase generation and validation method based on control flow graph (CFG). In addition, we propose effective fitness and coverage functions to guide testcase selection, innovatively taking into account the states of the Reorder Buffer (RoB) and the Branch Prediction Unit (BPU). This approach extends fitness and coverage metrics to the microarchitectural component level, improving vulnerability detection efficiency. Through BPUFuzzer testing, we identified a new transient execution pattern, termed the loop pattern, which can manifest as a new Spectre variant called Spectre-Loop. The main contributions of our work are:

- We propose a CFG-based testcase generation that encompasses all possible program control flows, including loop types that were not addressed in previous work.
- We propose an effective feedback method on fuzz testing to guide testcase selection. It significantly enhances the efficiency of uncovering transient execution vulnerabilities in complex hardware designs.
- The proposed fuzz tool successfully identified a new Spectre variant called Spectre-loop on Boom v3 CPU. Experiments also demonstrate that BPUFuzzer uncovers more Spectre types than previous work.

II. Related Work

The automated detection of transient execution vulnerabilities has gained significant attention. Prior work, such as Medusa [13], uses authenticated PoC snippets as test seeds. SpecFuzz [14] simulates branch misprediction in security-sensitive libraries. Absynthe [15] and Osiris [16] focus on finding specific side-channels. These approaches mainly test limited code snippets based on prior knowledge, rather than covering a broad range of code patterns and hardware states. Furthermore, these frameworks test only on post-silicon CPUs, limiting their ability to identify potential transient execution leaks in the pre-silicon design phase before the processor is taped out. In pre-silicon RTL-stage research, Introspectre [17] uses predefined code gadgets with a focus on meltdown-type vulnerabilities. DifuzzRTL [19] and SIGFuzz [20] apply a

*Corresponding author: dan@hit.edu.cn

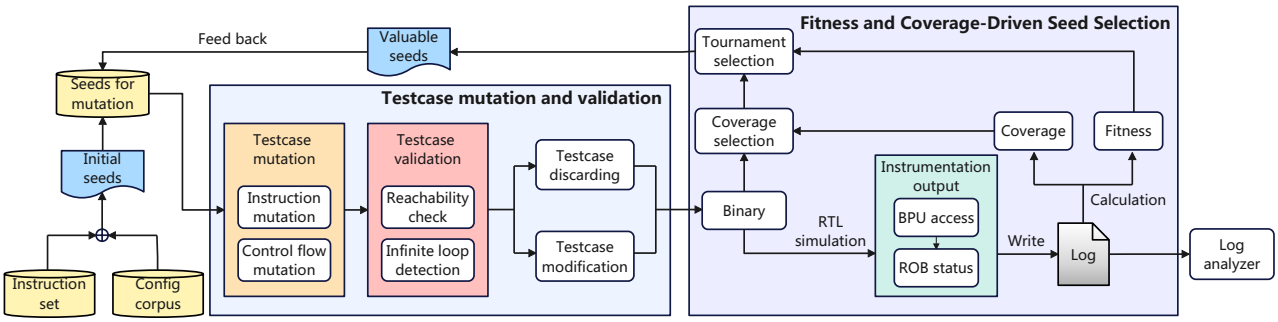


Fig. 1: Overview of BPUFuzzer

coverage based on control signals that drive the muxes to find CPU bugs and side channels. They focus on register states and ignore microarchitecture states, which is essential for transient execution. So their coverage definitions are unsuitable for detecting transient execution vulnerabilities. WhisperFuzz [21] focuses on timing behaviors changing by triggering different microarchitectural state transitions. In contrast, BPUFuzzer in-depth studies the states of microarchitectural components, including BPU and RoB. The coverage metrics formed by these states are used to search for testcases tend to trigger branching transient execution, significantly improving the efficiency of detecting vulnerabilities. SpecDoctor [18] generates testcases using CFG construction. To avoid invalid testcases in implementation, it enforces constraints that prevent loop generation. However, loop is an important control flow transfer pattern. Moreover, it does not provide coverage feedback mechanisms, limits its detection from complex hardware states. In contrast, BPUFuzzer is capable of discovering transient execution caused by any control flow transfer. Moreover, BPUFuzzer tailored coverage and fitness metrics optimized for transient execution. Owing to these two improvements, BPUFuzzer surpasses previous work in both the scope and efficiency of detecting transient execution vulnerabilities.

III. Overview of BPUFuzzer

In this section, we present an overview of the architecture of our transient execution vulnerability fuzz testing tool, BPUFuzzer. We consider the following threat model which aligns with the mainstream studies in discovering transient execution vulnerabilities such as Spectre and Meltdown:

(1) The attacker is a user without privilege, the victim is the privileged kernel. The attacker and the victim locate on the same logical core;

(2) The attacker can invoke the victim through system calls.

As shown in Figure 1, our tool includes two main modules to address the challenges outlined in Section I. The testcase mutation and validation module generates testcases for fuzz testing. To maximize the range of control flow patterns in testcase generation, We employ a modified CFG-based technique that includes instruction mutations and control flow mutations without any pattern constraints. A testcase validation submodule is also included to detect and avoid infinite loops. These methods enable us to produce a greater variety of control flow patterns compared with state-of-the-art tools.

The fitness and coverage-driven seed selection module select testcases with high tendency of transient execution and feed them back as seeds for mutation. Aimed at selecting valuable testcases efficiently, BPUFuzzer incorporates hardware information through instrumentation, including the status of BPU and RoB, which is closely related to transient execution. These pieces of information are logged and used to calculate the fitness function and testcase coverage. Specifically, testcases are first selected based on whether they cover new hardware states, as determined by coverage calculation, and then undergo tournament selection based on fitness value. Additionally, a log analyzer detects transient execution risks through the collected data and classifies them based on the originating microarchitectural component.

Sections IV and V will detail the working principles of the testcase mutation and validation module, as well as the fitness and coverage-driven seed selection module.

IV. Testcase Mutation and Validation

The testcase mutation and validation module aims to generate testcases that cover the maximum possible range of control flow patterns. This module consists of two components: mutation of instructions within a basic block and mutation of control flow instructions. To ensure that generated testcases maximize control flow coverage and execute to termination, a validation module checks each testcase after mutation.

A. Mutation Rule of a Basic Block

Figure 2 shows the structure of a basic block (BB) and all possible mutation behaviors. A BB must end with either a conditional or unconditional jump instruction, with no jump instructions allowed at any other position within the block. During mutation, various transformations may occur within the BB, including the insertion, removal, substitution of instructions, and modifications to jump instruction targets.

B. Control Flow Related Instructions on RISC-V

In RISC-V instruction sets, there are two types of instructions related to control flow transfer, unconditional jump instructions and conditional branch instructions, shown in Table I. A conditional branch adds two outgoing edges to the basic block in the CFG, while an unconditional jump only adds one outgoing edge.

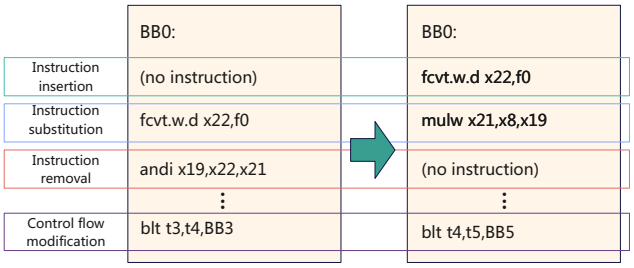


Fig. 2: Possible mutation operations on a basic block

TABLE I: Control Flow Related RISC-V Instructions

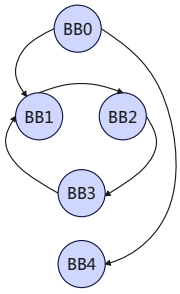
Inst	Description
beq, rs1, rs2, imm	if(rs1 == rs2) PC += imm
bne, rs1, rs2, imm	if(rs1 != rs2) PC += imm
blt, rs1, rs2, imm	if(rs1 $<$ rs2) PC += imm
bge, rs1, rs2, imm	if(rs1 $\geq$ rs2) PC += imm
bltu, rs1, rs2, imm	if(rs1 $<$ rs2) PC += imm
bgeu, rs1, rs2, imm	if(rs1 $\geq$ rs2) PC += imm
jal, rd, imm	rd = PC+4; PC += imm
jalr, rd, rs1, imm	rd = PC+4; PC = rs1 + imm

C. Testcase Validation

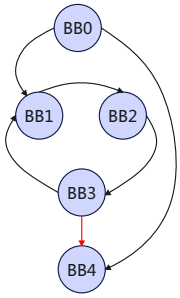
To maximize coverage of flow transfer scenarios in CFG generation, we do not restrict jump or branch targets during mutation. However, arbitrary jump combinations may create infinite loops, so we apply a validation process to detect and correct invalid testcases.

All loops in a CFG have the potential to cause infinite loops. Based on the types of jumps at the end of the BBs of a loop, we classify the causes of infinite loops into the following categories:

- **Unconditional jumps**: In this type, all BBs within the loop end with unconditional jumps. Consequently, once the control flow enters such a loop, it becomes impossible to exit. An example of this situation is illustrated in Figure 3(a), once the control flow jumps from BB0 to BB1, it will never exit. We will refer to this type of testcase as type 1 testcase in the following discussion.



(a) Infinite loop caused by unconditional jumps



(b) Infinite loop caused by conditional branches that always branch in the same direction

Fig. 3: Two types of invalid testcases

- **Conditional jumps with fixed direction**: Although at least one BB within these loops end with conditional jumps that could theoretically allow for an exit, arbitrary mutation operations may cause the direction of the conditional jump to remain unchanged, resulting in an infinite loop. As shown in Figure 3(b), although theoretically the unconditional jump at the end of BB3 could jump to BB4, improper instruction mutation may causes the value comparison between the two source registers of BB3's conditional jump to remain unchanged, preventing the jump from BB3 to BB4 from ever occurring(illustrated with a red arrow) and thus forming an infinite loop. We will refer to this type as type 2 testcase.

Notation	Description
BB.jmpIns	Last jump instruction of basic block `BB`.
BB.jmpIns.name	Name of the instruction.
BB.jmpIns.rs1	Source register 1 of the jump instruction.
BB.jmpIns.rs2	Source register 2 of the jump instruction.
instruction.dest	Destination register of arithmetic & logical instructions.

TABLE II: Notations used in Algorithm 1.

Algorithm 1: Type 2 testcases modification

```
Input : BBs containing conditional jump instructions in a cycle
Output: Modified BBs
foreach BB in BBs do
    if BB.jmpIns.name in {blt, bge, bltu, bgeu} then
        Set BB.jmpIns.rs1 to t1;
        Set BB.jmpIns.rs2 to t2;
        foreach instruction in BB do
            if instruction.dest in {t1, t2} then
                Delete instruction;
        ▷ Adjust the two source registers according to their current states;
        Insert instruction slti t3, t1, t2;
        Insert instruction xor t4, t3, 1;
        Insert instruction slli t3, t3, 6;
        Insert instruction slli t4, t4, 6;
        Insert instruction add t1, t1, t3;
        Insert instruction add t2, t2, t4;
        Insert instruction addi t1, t1, -32;
        Insert instruction addi t2, t2, -32;
    if BB.jmpIns.name in {beq, bne} then
        Set BB.jmpIns.rs1 to zero;
        Set BB.jmpIns.rs2 to t2;
        foreach instruction in BB do
            if instruction.dest = t2 then
                Delete instruction;
        Insert instruction srli t2, 1;
```

To identify and correct the two types of invalid testcases mentioned above, we employed a two-phase detection and modification approach after each round of mutation.

- **Discarding type 1 testcases**: We verify that all basic blocks reachable from the starting BB can also reach the terminating BB, any testcase that fails to meet this criterion will be discarded. Thus the type 1 invalid testcase can be avoided.

- **Modifying type 2 testcases**: After all type 1 testcases are discarded, each rest cycle in generated CFGs contains at least one conditional jump. In order to allow the direction of conditional jumps to change, we implement the following two measures: (I) Inserting instructions to adjust the two source registers according to their current states. For blt, bge, $bltu$ and $bgeu$, if the value of register a exceeds that of register b, we add instructions to reduce the value in register a and increase the value in register b, and vice versa. For bne and beq, since the probability of two 32-bit numbers being equal is extremely low, we implemented a specific approach: one source register is set to special zero register defined in RV32I [22], which always holds the value zero, while the other register is initialized with a non-zero value. Additionally, we insert a logical right shift instruction, consequently, the values in the two registers are always unequal initially, but after a finite number of loops, they will become equal. (II) Ensuring that no other instructions within the loop modify the contents of registers related to the jump. We deleted all instructions modifying any of the two resource registers before we insert the above instructions. The implementation can be found in Algorithm 1. The notations used in it are defined in Table II.

V. Fitness and Coverage-Driven Seed Selection

In this section, we present our implementation of the coverage and fitness functions and how they enhance our fuzz tool's ability to select more effective testcases.

A. Hardware Instrumentation

We collect essential microarchitectural data during the hardware instrumentation phase. The data that is interested is collected based on our study of the branching transient execution. The specific microarchitectural details are listed in Table III. We primarily gather information on BPU access and RoB status, which clearly reveals the behavior of transient execution and distinguishes them from normal program execution.

B. Design of Fitness Function and Coverage

Using the data in Table III, we designed a fitness function to estimate the likelihood of a testcase mutating into one that triggers transient execution. Additionally, we developed a coverage metric to efficiently search for testcases that tend to trigger transient execution from numerous testcase patterns corresponding to complex BPU states.

The formula of the fitness function is as follows:

$$fitness = mispredict\_count \times weight + btb\_hit\_count \quad (1)$$

The fitness function is a weighted sum of the misprediction count in the RoB and the BTB hit count. The misprediction count plays a critical role in triggering transient execution, while the BTB hit count is associated with the attacker's training phase in vulnerabilities like Spectre and its variants. In our testcase selection, greater emphasis is placed on the misprediction count, which is assigned a larger weight.

TABLE III: Hardware instrumentation information summary

uArch Comp.	Instrumentation Output	Description
RoB	mispredict_pc	The program counter of the mispredicted branch instruction.
	mispredict_ins_code	The instruction code of the mispredicted branch instruction.
	mispredict_target_addr	Mispredicted target address.
	mispredict_count	Total count of mispredictions.
BTB	wbtb_clock	Clock count when writing BTB.
	wbtb_bank_sel	Selection of bank when writing BTB, part of BTB index rule.
	wbtb_meta_bank_sel	Selection of bank when writing BTB meta data(tag and type), part of BTB index rule.
	wbtb_way	Selection of way when writing BTB, part of BTB index rule.
	wbtb_set	Selection of set when writing BTB, part of BTB index rule.
	wbtb_branch_offset	Offset of the branch ,i.e.target_pc-current_pc
	wbtb_branch_type	Type of branch instruction, i.e.conditional/unconditional
	wbtb_branch_tag	Tag associated with the branch, i.e. part of PC
	btb_hit_count	Count of BTB hits.
	btb_hit_set	The hit BTB set.
uBTB	ubtb_hit_count	Count of uBTB hits.
	ubtb_hit_tag	Tag in hit uBTB entry.
LOOP	loop_hit_count	Count of LOOP hits.
	loop_hit_set	Set of hit LOOP entry.
TAGE	tage_hit_count	Count of TAGE hits.
	tage_hit_table_idx	Index of hit TAGE table.
	tage_counter_count	Count of TAGE 3-bit counter.

For coverage, since each control flow transfer affects BTB states, we incorporate several branch prediction-related values into our coverage metric, which is shown in Algorithm 2.

In transient execution exploitation, the prediction training and triggering rely on the branch/jump instructions with specific instruction addresses, jump directions and target addresses. These values are stored in BTB and BTB meta table. To analyze these behaviors, we extract the relevant information and hash each corresponding piece of data in the log to generate binary feature values ($t.btb$, $t.btb\_meta$). $t.btb$ represents the features of jump directions and target addresses, while $t.btb\_meta$ represents the features of instruction addresses. If there is a new feature value, there is a control flow transfer instruction whose address or target address has changed. It may bring new transient execution possibility, reflected in an increase in test coverage.

Based on our fitness function and coverage, we select testcases to enhance defect detection effectiveness. First, we prioritize testcases that generate new coverage. If a testcase

Algorithm 2: Coverage Calculation

Input : A set of testcase and their logs
Output: whether the testcase is a new coverage

```
cov_list = empty set;
cov_sum = 0;
foreach t in test_case_set do
    t.btb = 0;
    t.btb_meta = 0;
    foreach line in test_case.log do
        btb_info = wbtb_set + wbtb_way +
          wbtb_branch_offset + wbtb_bank_sel;
        meta_info = wbtb_set + wbtb_way +
          wbtb_branch_tag + wbtb_branch_type +
          wbtb_meta_bank_sel;
        t.btb = t.btb xor btb_info;
        t.btb_meta = t.btb_meta xor meta_info;
    if (t.btb,t.btb_meta) not in cov_list then
        cov_list.insert(t.btb,t.btb_meta);
        t.coverage = true;
        cov_sum += 1;
    else
        t.coverage = false;
```

generates new coverage, it is added to the candidate set, while those without new coverage have a 5% probability of being included to prevent local optima. All testcases in the candidate set then undergo a tournament selection based on the fitness to form the next generation of testcases. This approach ensures that testing progressively covers more hardware states and that testcases increasingly tend to produce transient execution. The discovery of a new Spectre variant, called Spectre-Loop (detailed in Section VII), is owed to this methodology.

VI. Evaluation of BPUFuzzer

In this section, we present experiments that demonstrate the advantage of our proposed BPUFuzzer in terms of testcase generation pattern, coverage and vulnerability detection. We compare them with existing state-of-the-art tool, SpecDoctor.

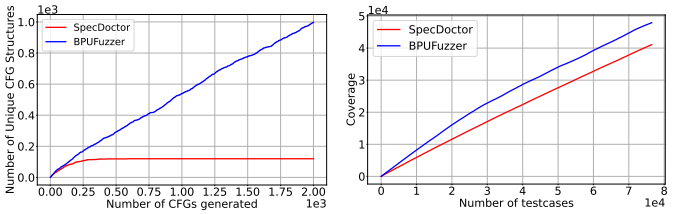
A. Experiment Setup

RISC-V CPU under test. We evaluated BPUFuzzer on the open-source out-of-order RISC-V CPU: Boom V3 [23]. It supports RV64G ISA which includes supervisor level instructions and user level instructions. It contains the out-of-order microarchitectural units including RoB and branch predictors.

Fuzz testing environment. We evaluated BPUFuzzer with a RTL simulator called Chipyard [24]. It is a modified Verilator simulator designed for Boom RISC-V CPU. All the fuzzing experiments are conducted on a machine equipped with an Intel(R) Xeon(R) E5-2670 v3 CPU at 2.30GHz with 64GB of RAM, which runs Ubuntu 22.04 LTS.

B. Testcase Generation Pattern

In section IV, we propose a testcase mutation and validation method designed to cover diverse control flow transfer patterns, corresponding to mutually non-isomorphic control flow graphs [25]. We verify the number of patterns generated and compare it with SpecDoctor.



(a) comparison of testcase pattern with SpecDoctor (b) comparison of test coverage with SpecDoctor

Fig. 4: Comparison with SpecDoctor

In Figure 4(a), we applied a basic block limit of 6 in the testcase generation process for both our tool and SpecDoctor. In SpecDoctor, basic blocks are connected by conditional jump instructions, and each block can only link to subsequent blocks, preventing loop structures. Under these constraints, SpecDoctor can generate only 120 unique testcase patterns, equivalent to 5 factorial. In contrast, our tool produces a greater variety of control flow transfer patterns, including testcases with loops that execute successfully.

C. Coverage

In Section V, we introduced a fitness function and coverage design based on microarchitectural state to guide testcase selection and mutation, and conducted a comparative experiment with SpecDoctor in Figure 4(b). Using the same hardware setup and the same testcase limit of 76,000, we tested on both tools respectively. SpecDoctor generated 34,916 testcases that do not generate new hardware states, which is unprofitable for transient execution detection. BPUFuzzer covered 6,846 more hardware states than SpecDoctor, leading by 16.7% in coverage. We also ran BPUFuzzer with and without coverage feedback under the same hardware setup for 40 hours. With coverage feedback, BPUFuzzer identified an average of 12.89 testcases with transient execution risks per minute, compared to 11.26 testcases per minute without coverage feedback. This represents a 14.5% improvement in transient execution detection efficiency, highlighting the benefits of coverage feedback.

D. Vulnerability Detection

We ran BPUFuzzer and SpecDoctor under the same hardware setup for a week and classified found vulnerability types following previous research [12]. The found vulnerability types comparison is shown in Table IV. BPUFuzzer successfully detected transient execution in micro BTB and LOOP while SpecDoctor do not.

In Boom v3, micro BTB is a prediction unit that stores local target addresses. It utilizes a fully-associative index method and only works when the corresponding BTB entry is empty. Spectre_uBTB needs to train the micro BTB while clearing the BTB entries. It requires the precise coordination of branch instructions in a wider range of instruction addresses than Spectre_BTB. However, the range of instruction addresses is relatively limited in SpecDoctor, makes it incapable of generating this vulnerability type.

TABLE IV: Found vulnerabilities summary and comparison with SpecDoctor

Category	Name	Description	BPUFuzzer	SpecDoctor
Bound Check Bypass	Spectre_BIM_IP	Exploitation of Bi-Modal Table.	✓	✓
	Spectre_uBTB_IP	Exploitation of micro BTB.	✓	×
	Spectre_TAGE_IP	Exploitation of TAGE.	✓	✓
	Spectre_TAGE_OP	Exploitation of TAGE with Table entry collision.	✓	✓
	Spectre_Loop_IP	Exploitation of Loop.	✓	×
	Spectre_Loop_OP	Exploitation of Loop with Table entry collision.	✓	×
Branch Target Injection	Spectre_uBTB_IP	Exploitation of micro BTB.	✓	×
	Spectre_BTB_IP	Exploitation of BTB.	✓	✓

VII. New Finding of BPUFuzzer: Spectre-Loop

During our analyzing on experiment results, we found some vulnerabilities have significantly large btb_hit_count in their fitness value. Through our further analysis, We found that they all have a loop structure. We eventually confirmed that they exploited a loop prediction optimization mechanism in Boom v3 and formed a new Spectre variant, termed Spectre-Loop.

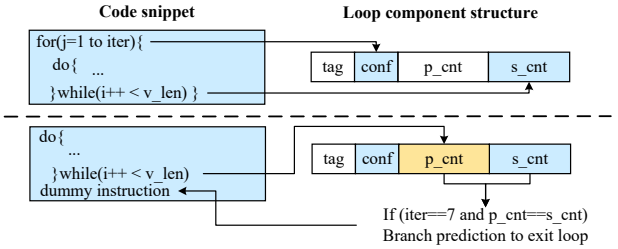
A. Details of Spectre-Loop

Figure 5(a) describes the working mechanism of LOOP, which predicts long loop exit timing of up to 1024. LOOP table has 4 parts: tag derived from the instruction's PC, is used to identify matching entries in the table. $iter$ is a three-bit saturating counter records times the loop is executed as a whole. p_cnt represents the present count of times the loop body has entered the pipeline during loop iteration, and it is updated each time the loop body enters the pipeline. s_cnt records the sum number of iterations after the loop code has fully completed execution. When the loop code runs again, if $iter$ reaches the threshold and p_cnt equals s_cnt, the prediction will indicate a loop exit. In summary, It is a prediction mechanism triggered by a loop code structure, this control flow pattern is not included in the testcase generation implementation of SpecDoctor. Thus SpecDoctor unable to detect transient execution caused by this prediction mechanism.

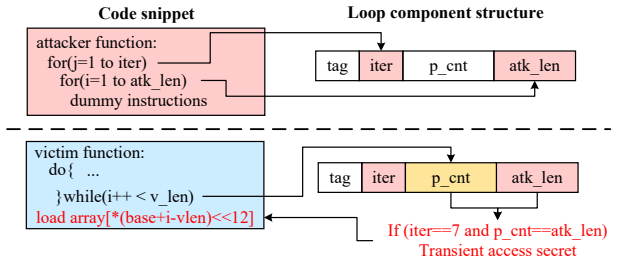
Figure 5(b) demonstrates the attack flow of spectre-loop. First, in the attacker phase, the loop is executed multiple times to train the s_cnt to the value atk_len. Then, in the trigger phase, the loop predict loop exit early after executing atk_len times. During speculative execution, this results in a transient out-of-bounds access on the offset of $atk\_len - v\_len$ (with $atk\_len < v\_len$). By controlling atk_len, an attacker can bypass the bound check and transiently access arbitrary locations, enabling access to specific secret data. The attacker can subsequently recover the transiently accessed secret data using a simple flush+reload technique.

B. Analysis of Spectre-Loop

Spectre-Loop successfully retrieved the secret of 1,024 bits with the error rate under 0.01%. Each bit leakage cost 220k clock cycles on the Chipyard simulator. The leakage rate is 4.9 Kb/s assuming a CPU clock rate of 1GHz.



(a) loop prediction optimization mechanism



(b) attack scenario of spectre-loop

Fig. 5: Loop prediction mechanism & Process of Spectre-loop

VIII. Conclusion

In this paper, we propose a pre-silicon fuzz testing tool, BPUFuzzer, to detect transient execution vulnerabilities. We propose a novel modified CFG-based test case generation method that outperforms all existing approaches in generating a broader range of control flow patterns, particularly by incorporating loop patterns that have not been considered in any prior work. Additionally, We develop novel coverage and fitness metrics tailored to the characteristics of transient execution, significantly enhancing the efficiency in detecting transient execution vulnerabilities. Finally, BPUFuzzer finds a newly discovered Spectre variant, termed Spectre-Loop, which exploits a previously overlooked prediction mechanism to perform a Bounds Check Bypass attack.

Acknowledgment

This work was supported by the Fundamental Research Funds for the Central Universities (Grant No. HIT.OCEF.2024012) and the National Natural Science Foundation of China (Grant No. U23B2041, U24A6009).

References

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1–19.

[2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 973–990.

[3] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. B. Abu-Ghazaleh, "Spectre returns! speculation attacks using the return stack buffer," in *12th USENIX Workshop on Offensive Technologies, WOOT 2018, Baltimore, MD, USA, August 13-14, 2018*, C. Rossow and Y. Younan, Eds. USENIX Association, 2018.

[4] J. Wikner and K. Razavi, "RETBLEED: arbitrary speculative code execution with return instructions," in *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, K. R. B. Butler and K. Thomas, Eds. USENIX Association, 2022, pp. 3825–3842.

[5] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. M. Tullsen, "Pathfinder: High-resolution control-flow attacks exploiting the conditional branch predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafrir, Eds. ACM, 2024, pp. 770–784.

[6] S. van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "RIDL: rogue in-flight data load," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 88–105.

[7] M. Schwarz, M. Lipp, D. Moghimi, J. V. Bulck, J. Stecklina, T. Prescher, and D. Gruss, "Zombieload: Cross-privilege-boundary data sampling," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, L. Cavallaro, J. Kinder, X. Wang, and J. Katz, Eds. ACM, 2019, pp. 753–768.

[8] J. Van Bulck, D. Moghimi, M. Schwarz, M. Lippi, M. Minkin, D. Genkin, Y. Yarom, B. Sunar, D. Gruss, and F. Piessens, "Lvi: Hijacking transient execution through microarchitectural load value injection," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 54–72.

[9] G. Maisuradze and C. Rossow, "ret2spec: Speculative execution using return stack buffers," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2109–2122.

[10] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, "Branch history injection: On the effectiveness of hardware mitigations against Cross-Privilege spectre-v2 attacks," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 971–988.

[11] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, "Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. D. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 1451–1468.

[12] C. Canella, J. V. Bulck, M. Schwarz, M. Lipp, B. von Berg, P. Ortner, F. Piessens, D. Evtyushkin, and D. Gruss, "A systematic evaluation of transient execution attacks and defenses," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 249–266.

[13] D. Moghimi, M. Lipp, B. Sunar, and M. Schwarz, "Medusa: Microarchitectural data leakage via automated attack synthesis," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1427–1444.

[14] O. Oleksenko, B. Trach, M. Silberstein, and C. Fetzer, "SpecFuzz: Bringing spectre-type vulnerabilities to the surface," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1481–1498.

[15] B. Gras, C. Giuffrida, M. Kurth, H. Bos, and K. Razavi, "Absynthe: Automatic blackbox side-channel synthesis on commodity microarchitectures," in *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.

[16] D. Weber, A. Ibrahim, H. Nemati, M. Schwarz, and C. Rossow, "Osiris: Automated discovery of microarchitectural side channels," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1415–1432. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/weber

[17] M. Ghaniyoun, K. Barber, Y. Zhang, and R. Teodorescu, "INTROSPECTRE: A pre-silicon framework for discovery and analysis of transient execution vulnerabilities," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 874–887.

[18] J. Hur, S. Song, S. Kim, and B. Lee, "Specdoctor: Differential fuzz testing to find transient execution vulnerabilities," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1473–1487.

[19] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, "Difuzzrtl: Differential fuzz testing to find cpu bugs," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1286–1303.

[20] C. Rajapaksha, L. Delshadtehrani, M. Egele, and A. Joshi, "Sigfuzz: A framework for discovering microarchitectural timing side channels," in *2023 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–6.

[21] P. Borkar, C. Chen, M. Rostami, N. Singh, R. Kande, A. Sadeghi, C. Rebeiro, and J. Rajendran, "Whisperfuzz: White-box fuzzing for detecting and locating timing vulnerabilities in processors," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024.

[22] A. Waterman and K. Asanović, "The risc-v instruction set manual, volume i: User-level isa, document version 20191214-draft," *RISC-V Foundation*, December 2019.

[23] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "Sonicboom: The 3rd generation berkeley out-of-order machine," May 2020.

[24] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[25] L. Babai, "Group, graphs, algorithms: the graph isomorphism problem," in *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*. World Scientific, 2018, pp. 3319–3336.